



Exercise on Data Preprocessing & Visualization

PROBLEM STATEMENT You are required to clean, preprocess, and visualize the retail transactional dataset to uncover hidden insights. The final objective is to create a clean dataset ready for analysis and generate visual dashboards that highlight customer and sales trends.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data = {
    'TransactionID': [1001, 1002, 1003, 1004, 1005, 1007, 1008],
    'CustomerID': ['C101', 'C102', 'C103', 'C104', 'C105', 'C107', 'C108'],
    'Gender': ['Male', 'female', 'M', 'female', 'MALE', 'Male', 'Other', 'female'],
    'Age': [25, 34, np.nan, 45, 22, 22, 61, 39],
    'City': ['Vadodara', 'Ahmedabad', np.nan, 'Surat', 'Vadodara', 'Vadodara', 'Rajkot', 'Ahmedabad'],
    'ProductCategory': ['Electronics', 'Fashion', 'Groceries', 'Furniture', 'Electronics', 'Electronics', 'Fashion', 'Groceries'],
    'Quantity': [2, 3, 1, 0, 2, 2, 4, -1],
    'Price': [25000, 1200, 500, 15000, 30000, 30000, 1800, 600],
    'TotalAmount': [50000, 3600, 500, np.nan, 60000, 60000, 7200, np.nan],
    'PurchaseDate': [
        '2026-01-15',
        '2026-02-10',
        '2026-02-25',
        '2026-03-12',
        '2026-03-20',
        '2026-03-20',
        '2026-04-05',
        '2026-04-18'
    ],
    'PaymentMode': ['UPI', 'Card', 'Cash', 'UPI', 'Wallet', 'Wallet', 'Card', 'UPI']
}

df = pd.DataFrame(data)
df
```

Output:

	TransactionID	CustomerID	Gender	Age	City	ProductCategory	Quantity	Price	TotalAmount	PurchaseDate	PaymentMode
0	1001	C101	Male	25.0	Vadodara	Electronics	2	25000	50000.0	2026-01-15	UPI
1	1002	C102	female	34.0	Ahmedabad	Fashion	3	1200	3600.0	2026-02-10	Card
2	1003	C103	M	NaN	NaN	Groceries	1	500	500.0	2026-02-25	Cash
3	1004	C104	Female	45.0	Surat	Furniture	0	15000	NaN	2026-03-12	UPI
4	1005	C105	MALE	22.0	Vadodara	Electronics	2	30000	60000.0	2026-03-20	Wallet
5	1005	C105	Male	22.0	Vadodara	Electronics	2	30000	60000.0	2026-03-20	Wallet
6	1007	C107	Other	61.0	Rajkot	Fashion	4	1800	7200.0	2026-04-05	Card
7	1008	C108	Female	39.0	Ahmedabad	Groceries	-1	600	NaN	2026-04-18	UPI



PART A: DATA PREPROCESSING TASKS

1. Data Inspection

- Load the dataset and check for size, structure, and column details.
- Identify missing values, duplicates, and inconsistent data.

Input Command

```
print("Shape of dataset:", df.shape)
```

Output

```
Shape of dataset: (8, 11)
```

Input Command

```
df.info()
```

Output

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 11 columns):
#   Column          Non-Null Count  Dtype  
---  -
0   TransactionID    8 non-null     int64  
1   CustomerID      8 non-null     object  
2   Gender          8 non-null     object  
3   Age             7 non-null     float64 
4   City            7 non-null     object  
5   ProductCategory 8 non-null     object  
6   Quantity        8 non-null     int64  
7   Price           8 non-null     int64  
8   TotalAmount     6 non-null     float64 
9   PurchaseDate    8 non-null     object  
10  PaymentMode     8 non-null     object  
dtypes: float64(2), int64(3), object(6)
memory usage: 836.0+ bytes
```



Input Command

```
df.head()
```

Output

	TransactionID	CustomerID	Gender	Age	City	ProductCategory	Quantity	Price	TotalAmount	PurchaseDate	PaymentMode
0	1001	C101	Male	25.0	Vadodara	Electronics	2	25000	50000.0	2026-01-15	UPI
1	1002	C102	female	34.0	Ahmedabad	Fashion	3	1200	3600.0	2026-02-10	Card
2	1003	C103	M	NaN	NaN	Groceries	1	500	500.0	2026-02-25	Cash
3	1004	C104	Female	45.0	Surat	Furniture	0	15000	NaN	2026-03-12	UPI
4	1005	C105	MALE	22.0	Vadodara	Electronics	2	30000	60000.0	2026-03-20	Wallet

Input Command

```
print(df.columns)
```

Output

```
Index(['TransactionID', 'CustomerID', 'Gender', 'Age', 'City',  
      'ProductCategory', 'Quantity', 'Price', 'TotalAmount', 'PurchaseDate',  
      'PaymentMode'],  
      dtype='object')
```

Input Command

```
print(df.isnull().sum())
```

Output

```
TransactionID    0  
CustomerID      0  
Gender          0  
Age             1  
City            1  
ProductCategory 0  
Quantity        0  
Price           0  
TotalAmount     2  
PurchaseDate    0  
PaymentMode     0  
dtype: int64
```



Input Command

```
print("\nDuplicate Rows:")  
print(df.duplicated().sum())
```

Output

```
Duplicate Rows:  
0
```

2. Handling Missing Data

- Replace missing Age with mean/median.
- Replace missing City with most frequent value.
- Drop rows where critical fields like TransactionID or ProductCategory are missing.

Input Command

```
df['Age'] = df['Age'].fillna(df['Age'].median())  
df
```

Output

	TransactionID	CustomerID	Gender	Age	City	ProductCategory	Quantity	Price	TotalAmount	PurchaseDate	PaymentMode
0	1001	C101	Male	25.0	Vadodara	Electronics	2	25000	50000.0	2026-01-15	UPI
1	1002	C102	female	34.0	Ahmedabad	Fashion	3	1200	3600.0	2026-02-10	Card
2	1003	C103	M	34.0	Vadodara	Groceries	1	500	500.0	2026-02-25	Cash
3	1004	C104	Female	45.0	Surat	Furniture	0	15000	NaN	2026-03-12	UPI
4	1005	C105	MALE	22.0	Vadodara	Electronics	2	30000	60000.0	2026-03-20	Wallet
5	1005	C105	Male	22.0	Vadodara	Electronics	2	30000	60000.0	2026-03-20	Wallet
6	1007	C107	Other	61.0	Rajkot	Fashion	4	1800	7200.0	2026-04-05	Card
7	1008	C108	Female	39.0	Ahmedabad	Groceries	-1	600	NaN	2026-04-18	UPI



Input command

```
df['City'] = df['City'].fillna(df['City'].mode()[0])  
df
```

Output

	TransactionID	CustomerID	Gender	Age	City	ProductCategory	Quantity	Price	TotalAmount	PurchaseDate	PaymentMode
0	1001	C101	Male	25.0	Vadodara	Electronics	2	25000	50000.0	2026-01-15	UPI
1	1002	C102	female	34.0	Ahmedabad	Fashion	3	1200	3600.0	2026-02-10	Card
2	1003	C103	M	34.0	Vadodara	Groceries	1	500	500.0	2026-02-25	Cash
3	1004	C104	Female	45.0	Surat	Furniture	0	15000	NaN	2026-03-12	UPI
4	1005	C105	MALE	22.0	Vadodara	Electronics	2	30000	60000.0	2026-03-20	Wallet
5	1005	C105	Male	22.0	Vadodara	Electronics	2	30000	60000.0	2026-03-20	Wallet
6	1007	C107	Other	61.0	Rajkot	Fashion	4	1800	7200.0	2026-04-05	Card
7	1008	C108	Female	39.0	Ahmedabad	Groceries	-1	600	NaN	2026-04-18	UPI

Input Command

```
df = df.dropna(subset=['TransactionID', 'ProductCategory'])  
df
```

Output

	TransactionID	CustomerID	Gender	Age	City	ProductCategory	Quantity	Price	TotalAmount	PurchaseDate	PaymentMode
0	1001	C101	Male	25.0	Vadodara	Electronics	2	25000	50000.0	2026-01-15	UPI
1	1002	C102	female	34.0	Ahmedabad	Fashion	3	1200	3600.0	2026-02-10	Card
2	1003	C103	M	NaN	NaN	Nan	1	500	500.0	2026-02-25	Cash
3	1004	C104	Female	45.0	Surat	Furniture	0	15000	NaN	2026-03-12	UPI
4	1005	C105	MALE	22.0	Vadodara	Electronics	2	30000	60000.0	2026-03-20	Wallet
5	1005	C105	Male	22.0	Vadodara	Electronics	2	30000	60000.0	2026-03-20	Wallet
6	1007	C107	Other	61.0	Rajkot	Fashion	4	1800	7200.0	2026-04-05	Card
7	1008	C108	Female	39.0	Ahmedabad	Groceries	-1	600	NaN	2026-04-18	UPI

3. Data Cleaning - Remove duplicate transactions.

- Standardize categorical values (e.g., 'm', 'Male', 'MALE' → 'Male').
- Correct negative or zero values in Quantity or Price.

Input Command

```
df['Gender'] = df['Gender'].replace({  
    'M': 'Male',  
    'MALE': 'Male',  
    'male': 'Male',  
    'female': 'Female',  
    'FEMALE': 'Female',  
    'f': 'Female'  
})  
df
```

Output

	TransactionID	CustomerID	Gender	Age	City	ProductCategory	Quantity	Price	TotalAmount	PurchaseDate	PaymentMode
0	1001	C101	Male	25.0	Vadodara	Electronics	2	25000	50000.0	2026-01-15	UPI
1	1002	C102	Female	34.0	Ahmedabad	Fashion	3	1200	3600.0	2026-02-10	Card
2	1003	C103	Male	NaN	NaN	Nan	1	500	500.0	2026-02-25	Cash
3	1004	C104	Female	45.0	Surat	Furniture	0	15000	NaN	2026-03-12	UPI
4	1005	C105	Male	22.0	Vadodara	Electronics	2	30000	60000.0	2026-03-20	Wallet
5	1005	C105	Male	22.0	Vadodara	Electronics	2	30000	60000.0	2026-03-20	Wallet
6	1007	C107	Other	61.0	Rajkot	Fashion	4	1800	7200.0	2026-04-05	Card
7	1008	C108	Female	39.0	Ahmedabad	Groceries	-1	600	NaN	2026-04-18	UPI

```
print(df['Gender'].value_counts())  
df
```

```
Gender  
Male      4  
Female    3  
Other     1  
Name: count, dtype: int64
```



4. Feature Engineering

- Derive TotalAmount if missing.
- Extract Month and DayOfWeek from PurchaseDate.
- Create an AgeGroup column (e.g., 18-25, 26-40, 41-60, 60+).

Input Command

```
df['TotalAmount'] = df['Quantity'] * df['Price']  
df
```

Output

	TransactionID	CustomerID	Gender	Age	City	ProductCategory	Quantity	Price	TotalAmount	PurchaseDate	PaymentMode	Month	DayOfWeek
0	1001	C101	Male	25.0	Vadodara	Electronics	2	25000	50000	2026-01-15	UPI	January	Thursday
1	1002	C102	Female	34.0	Ahmedabad	Fashion	3	1200	3600	2026-02-10	Card	February	Tuesday
2	1003	C103	Male	NaN	NaN	Nan	1	500	500	2026-02-25	Cash	February	Wednesday
3	1004	C104	Female	45.0	Surat	Furniture	2	15000	30000	2026-03-12	UPI	March	Thursday
4	1005	C105	Male	22.0	Vadodara	Electronics	2	30000	60000	2026-03-20	Wallet	March	Friday
5	1005	C105	Male	22.0	Vadodara	Electronics	2	30000	60000	2026-03-20	Wallet	March	Friday
6	1007	C107	Other	61.0	Rajkot	Fashion	4	1800	7200	2026-04-05	Card	April	Sunday
7	1008	C108	Female	39.0	Ahmedabad	Groceries	2	600	1200	2026-04-18	UPI	April	Saturday

Input Command

```
df['PurchaseDate'] = pd.to_datetime(df['PurchaseDate'])  
print(df['PurchaseDate'].dtype)  
df
```

Output

datetime64[ns]													
	TransactionID	CustomerID	Gender	Age	City	ProductCategory	Quantity	Price	TotalAmount	PurchaseDate	PaymentMode	Month	DayOfWeek
0	1001	C101	Male	25.0	Vadodara	Electronics	2	25000	50000	2026-01-15	UPI	January	Thursday
1	1002	C102	Female	34.0	Ahmedabad	Fashion	3	1200	3600	2026-02-10	Card	February	Tuesday
2	1003	C103	Male	NaN	NaN	Nan	1	500	500	2026-02-25	Cash	February	Wednesday
3	1004	C104	Female	45.0	Surat	Furniture	2	15000	30000	2026-03-12	UPI	March	Thursday
4	1005	C105	Male	22.0	Vadodara	Electronics	2	30000	60000	2026-03-20	Wallet	March	Friday
5	1005	C105	Male	22.0	Vadodara	Electronics	2	30000	60000	2026-03-20	Wallet	March	Friday
6	1007	C107	Other	61.0	Rajkot	Fashion	4	1800	7200	2026-04-05	Card	April	Sunday
7	1008	C108	Female	39.0	Ahmedabad	Groceries	2	600	1200	2026-04-18	UPI	April	Saturday



Input

```
: df['Month'] = df['PurchaseDate'].dt.month_name()  
  
df['DayOfWeek'] = df['PurchaseDate'].dt.day_name()  
df
```

Output

```
]:
```

	TransactionID	CustomerID	Gender	Age	City	ProductCategory	Quantity	Price	TotalAmount	PurchaseDate	PaymentMode	Month	DayOfWeek
0	1001	C101	Male	25.0	Vadodara	Electronics	2	25000	50000	2026-01-15	UPI	January	Thursday
1	1002	C102	Female	34.0	Ahmedabad	Fashion	3	1200	3600	2026-02-10	Card	February	Tuesday
2	1003	C103	Male	NaN	NaN	Nan	1	500	500	2026-02-25	Cash	February	Wednesday
3	1004	C104	Female	45.0	Surat	Furniture	2	15000	30000	2026-03-12	UPI	March	Thursday
4	1005	C105	Male	22.0	Vadodara	Electronics	2	30000	60000	2026-03-20	Wallet	March	Friday
5	1005	C105	Male	22.0	Vadodara	Electronics	2	30000	60000	2026-03-20	Wallet	March	Friday
6	1007	C107	Other	61.0	Rajkot	Fashion	4	1800	7200	2026-04-05	Card	April	Sunday
7	1008	C108	Female	39.0	Ahmedabad	Groceries	2	600	1200	2026-04-18	UPI	April	Saturday

Input Command

```
bins = [0, 25, 40, 60, np.inf]
labels = ['18-25', '26-40', '41-60', '60+']

df['AgeGroup'] = pd.cut(
    df['Age'],
    bins=bins,
    labels=labels,
    right=True
)
print(df[['Age', 'AgeGroup']])
print(df)
```

Output

The image shows a JupyterLab interface in a web browser. The top bar displays the file explorer with a folder named 'Untitled'. The main area contains a Jupyter Notebook with two dataframes. The first dataframe, 'AgeGroup', has columns 'Age' and 'AgeGroup'. The second dataframe, 'TransactionID', has columns 'TransactionID', 'CustomerID', 'Gender', 'Age', 'City', and 'ProductCategory'. The third dataframe, 'Quantity', has columns 'Quantity', 'Price', 'TotalAmount', 'PurchaseDate', 'PaymentMode', 'Month', and 'DayOfWeek'. The fourth dataframe, 'AgeGroup', has columns 'AgeGroup'. The interface includes a top bar with file explorer, a toolbar with icons for file operations, and a bottom status bar showing the current environment as Python [conda env:base].

5. Encoding & Transformation

- Encode categorical variables for analysis (e.g., Gender, City).
- Normalize numerical columns (Age, Price, TotalAmount) if required.

Input Command

```
df_encoded = pd.get_dummies(
    df,
    columns=['Gender', 'City', 'ProductCategory', 'PaymentMode'],
    dtype=int
)

print(df_encoded)
```

Output

	TransactionID	CustomerID	Age	Quantity	Price	TotalAmount	PurchaseDate	\
0	1001	C101	25.0	2	25000	50000	2026-01-15	
1	1002	C102	34.0	3	1200	3600	2026-02-10	
2	1003	C103	NaN	1	500	500	2026-02-25	
3	1004	C104	45.0	2	15000	30000	2026-03-12	
4	1005	C105	22.0	2	30000	60000	2026-03-20	
5	1005	C105	22.0	2	30000	60000	2026-03-20	
6	1007	C107	61.0	4	1800	7200	2026-04-05	
7	1008	C108	39.0	2	600	1200	2026-04-18	

	Month	DayOfWeek	AgeGroup	...	City_Vadodara	\
0	January	Thursday	18-25	...	1	
1	February	Tuesday	26-40	...	0	
2	February	Wednesday	NaN	...	0	
3	March	Thursday	41-60	...	0	
4	March	Friday	18-25	...	1	
5	March	Friday	18-25	...	1	
6	April	Sunday	60+	...	0	
7	April	Saturday	26-40	...	0	

	ProductCategory_Electronics	ProductCategory_Fashion	\
0	1	0	
1	0	1	
2	0	0	
3	0	0	
4	1	0	
5	1	0	
6	0	1	
7	0	0	



```
ProductCategory_Furniture ProductCategory_Groceries ProductCategory_Nan \
0 0 0 0
1 0 0 0
2 0 0 1
3 1 0 0
4 0 0 0
5 0 0 0
6 0 0 0
7 0 1 0

PaymentMode_Card PaymentMode_Cash PaymentMode_UPI PaymentMode_Wallet
0 0 0 1 0
1 1 0 0 0
2 0 1 0 0
3 0 0 1 0
4 0 0 0 1
5 0 0 0 1
6 1 0 0 0
7 0 0 1 0

[8 rows x 26 columns]
```

Input

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

df_encoded[['Age', 'Price', 'TotalAmount']] = scaler.fit_transform(
    df_encoded[['Age', 'Price', 'TotalAmount']]
)

df_encoded.head()
```

Output

	TransactionID	CustomerID	Age	Quantity	Price	TotalAmount	PurchaseDate	Month	DayOfWeek	AgeGroup	...	City_Vadodara	ProductCategory_Elect
0	1001	C101	0.076923	2	0.830508	0.831933	2026-01-15	January	Thursday	18-25	...	1	
1	1002	C102	0.307692	3	0.023729	0.052101	2026-02-10	February	Tuesday	26-40	...	0	
2	1003	C103	NaN	1	0.000000	0.000000	2026-02-25	February	Wednesday	NaN	...	0	
3	1004	C104	0.589744	2	0.491525	0.495798	2026-03-12	March	Thursday	41-60	...	0	
4	1005	C105	0.000000	2	1.000000	1.000000	2026-03-20	March	Friday	18-25	...	1	

5 rows x 26 columns



ProductCategory_Electronics	ProductCategory_Fashion	ProductCategory_Furniture	ProductCategory_Groceries	ProductCategory_Nan	PaymentMode_Card
1	0	0	0	0	0
0	1	0	0	0	0
0	0	0	0	1	0
0	0	1	0	0	0
1	0	0	0	0	0

PaymentMode_Cash	PaymentMode_UPI	PaymentMode_Wallet
0	1	0
0	0	0
1	0	0
0	1	0
0	0	1

6. Final Verification

- Ensure no missing/invalid values remain.
- Save the cleaned dataset as Retail_Cleaned.csv.

Input Command

```
print("Missing Values:")
print(df.isnull().sum())

print("\nDuplicate Transactions:")
print(df['TransactionID'].duplicated().sum())

print("\nInvalid Quantity:")
print((df['Quantity'] <= 0).sum())

print("\nInvalid Price:")
print((df['Price'] <= 0).sum())
```



Output

Missing Values:

```
TransactionID      0
CustomerID         0
Gender             0
Age               1
City              1
ProductCategory    0
Quantity          0
Price             0
TotalAmount        2
PurchaseDate       0
PaymentMode        0
dtype: int64
```

Duplicate Transactions:

```
1
```

Invalid Quantity:

```
2
```

Invalid Price:

```
0
```

Input Command

```
[8]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data = {
    'TransactionID': [1001, 1002, 1003, 1004, 1005, 1005, 1007, 1008],
    'CustomerID': ['C101', 'C102', 'C103', 'C104', 'C105', 'C105', 'C107', 'C108'],
    'Gender': ['Male', 'female', 'M', 'Female', 'MALE', 'Male', 'Other', 'Female'],
    'Age': [25, 34, np.nan, 45, 22, 22, 61, 39],
    'City': ['Vadodara', 'Ahmedabad', np.nan, 'Surat', 'Vadodara', 'Vadodara', 'Rajkot', 'Ahmedabad'],
    'ProductCategory': ['Electronics', 'Fashion', 'Groceries', 'Furniture', 'Electronics', 'Electronics', 'Fashion', 'Groceries'],
    'Quantity': [2, 3, 1, 0, 2, 2, 4, -1],
    'Price': [25000, 1200, 500, 15000, 30000, 30000, 1800, 600],
    'TotalAmount': [50000, 3600, 500, np.nan, 60000, 60000, 7200, np.nan],
    'PurchaseDate': [
        '2026-01-15',
        '2026-02-10',
        '2026-02-25',
        '2026-03-12',
        '2026-03-20',
        '2026-03-20',
        '2026-04-05',
        '2026-04-18'
    ],
    'PaymentMode': ['UPI', 'Card', 'Cash', 'UPI', 'Wallet', 'Wallet', 'Card', 'UPI']
}

df = pd.DataFrame(data)

df
df.to_csv('Retail_Cleaned.csv', index=False)
print("Cleaned dataset saved successfully!")

Cleaned dataset saved successfully!
```



Output

localhost:8888/edit/DSE-practice/Retail_Cleaned.csv?

Jupyter Retail_Cleaned.csv Last Checkpoint: 4 minutes ago

File Edit View Settings Help

Delimiter: .

	TransactionID	CustomerID	Gender	Age	City	ProductCategory	Quantity	Price
1	1001	C101	Male	25.0	Vadodara	Electronics	2	2500
2	1002	C102	female	34.0	Ahmedabad	Fashion	3	1200
3	1003	C103	M			Groceries	1	500
4	1004	C104	Female	45.0	Surat	Furniture	0	15000
5	1005	C105	MALE	22.0	Vadodara	Electronics	2	30000
6	1005	C105	Male	22.0	Vadodara	Electronics	2	30000
7	1007	C107	Other	61.0	Rajkot	Fashion	4	1800
8	1008	C108	Female	39.0	Ahmedabad	Groceries	-1	600

Price	TotalAmount	PurchaseDate
25000	50000.0	2026-01-15
1200	3600.0	2026-02-10
500	500.0	2026-02-25
15000		2026-03-12
30000	60000.0	2026-03-20
30000	60000.0	2026-03-20
1800	7200.0	2026-04-05
600		2026-04-18

PART B: DATA VISUALIZATION TASKS.

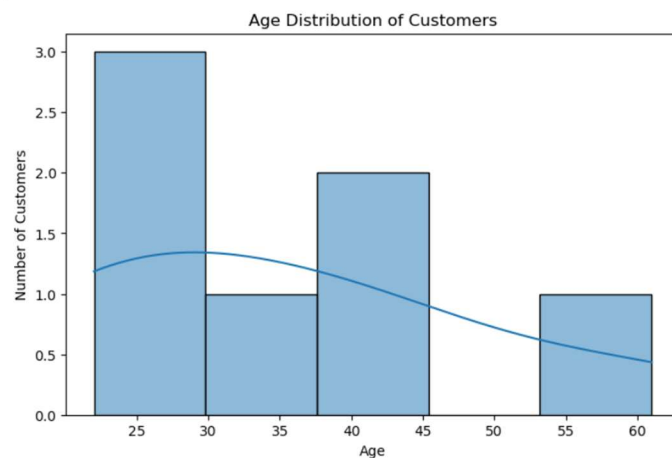
1. Customer Demographics

- Age distribution of customers.
- Gender distribution.
- Customers by city (Top 10 cities).

Input Command

```
plt.figure(figsize=(8,5))  
  
sns.histplot(df['Age'], bins=5, kde=True)  
  
plt.title('Age Distribution of Customers')  
plt.xlabel('Age')  
plt.ylabel('Number of Customers')  
  
plt.show()
```

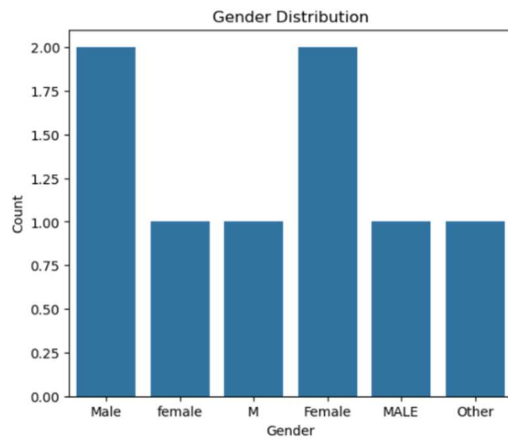
Output



Input Command

```
plt.figure(figsize=(6,5))  
sns.countplot(data=df, x='Gender')  
plt.title('Gender Distribution')  
plt.xlabel('Gender')  
plt.ylabel('Count')  
plt.show()
```

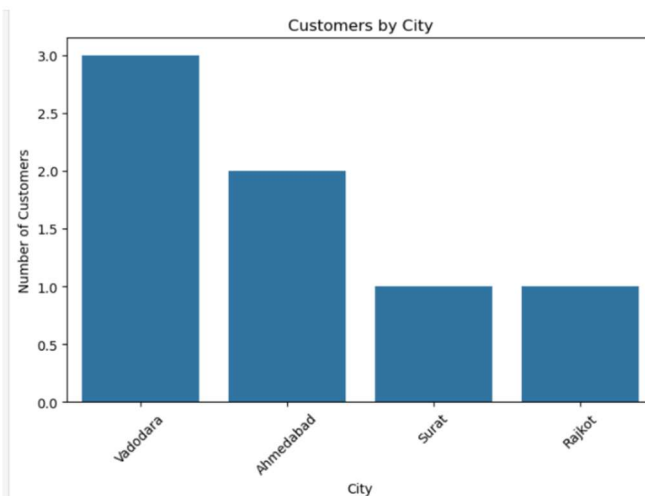
Output



Input Command

```
city_counts = df['City'].value_counts()
plt.figure(figsize=(8,5))
sns.barplot(
    x=city_counts.index,
    y=city_counts.values
)
plt.title('Customers by City')
plt.xlabel('City')
plt.ylabel('Number of Customers')
plt.xticks(rotation=45)
plt.show()
```

Output



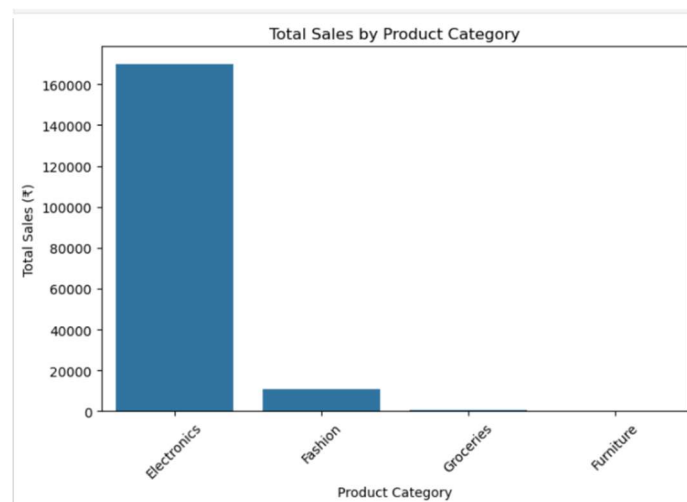
2. Sales Insights

- Total sales by product category.
- Monthly sales trend (line chart).
- Payment mode usage (pie chart).

Input Command

```
category_sales = df.groupby(  
    'ProductCategory'  
)['TotalAmount'].sum().sort_values(ascending=False)  
plt.figure(figsize=(8,5))  
sns.barplot(  
    x=category_sales.index,  
    y=category_sales.values  
)  
plt.title('Total Sales by Product Category')  
plt.xlabel('Product Category')  
plt.ylabel('Total Sales (₹)')  
plt.xticks(rotation=45)  
plt.show()
```

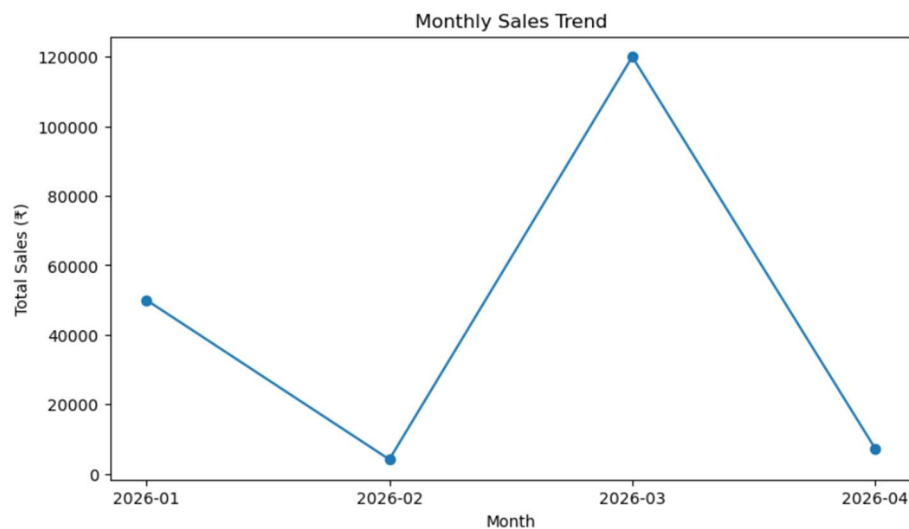
Output



Input Command

```
df['PurchaseDate'] = pd.to_datetime(df['PurchaseDate'], errors='coerce')
monthly_sales = df.groupby(
    df['PurchaseDate'].dt.to_period('M')
)['TotalAmount'].sum()
monthly_sales.index = monthly_sales.index.astype(str)
plt.figure(figsize=(9,5))
plt.plot(
    monthly_sales.index,
    monthly_sales.values,
    marker='o'
)
plt.title('Monthly Sales Trend')
plt.xlabel('Month')
plt.ylabel('Total Sales (₹)')
plt.xticks(rotation=45)
plt.show()
```

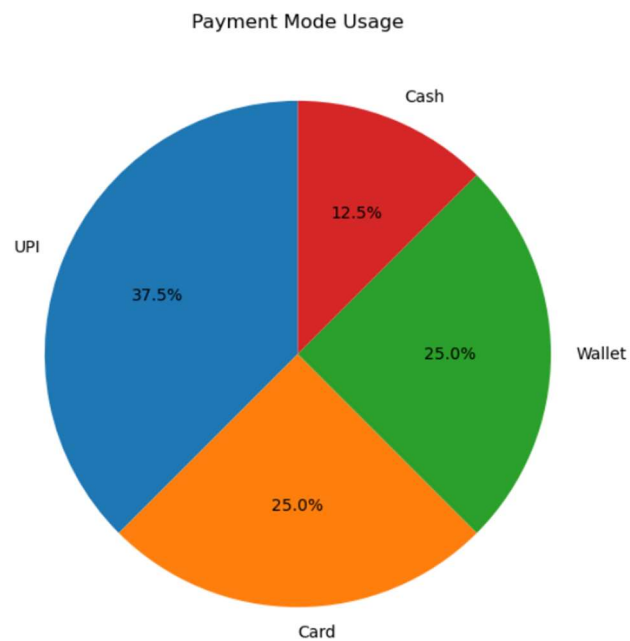
Output



Input Command

```
: payment_counts = df['PaymentMode'].value_counts()
plt.figure(figsize=(7,7))
plt.pie(
    payment_counts.values,
    labels=payment_counts.index,
    autopct='%1.1f%%',
    startangle=90
)
plt.title('Payment Mode Usage')
plt.show()
```

Output



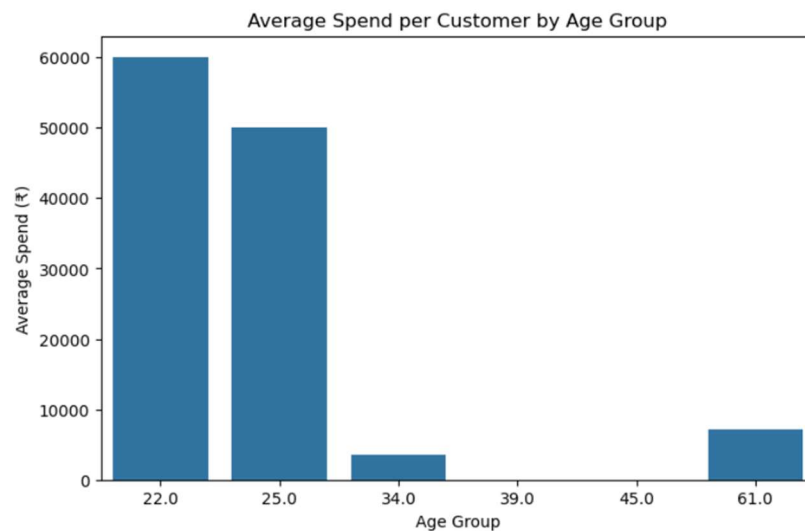
3. Advanced Insights

- Average spend per customer by age group.
- City-wise revenue contribution (bar chart).
- Heatmap of product category vs payment mode.

Input Command

```
age_group_spend = df.groupby(  
    'Age',  
    observed=True  
)['TotalAmount'].mean()  
plt.figure(figsize=(8,5))  
sns.barplot(  
    x=age_group_spend.index,  
    y=age_group_spend.values  
)  
plt.title('Average Spend per Customer by Age Group')  
plt.xlabel('Age Group')  
plt.ylabel('Average Spend (₹)')  
plt.show()
```

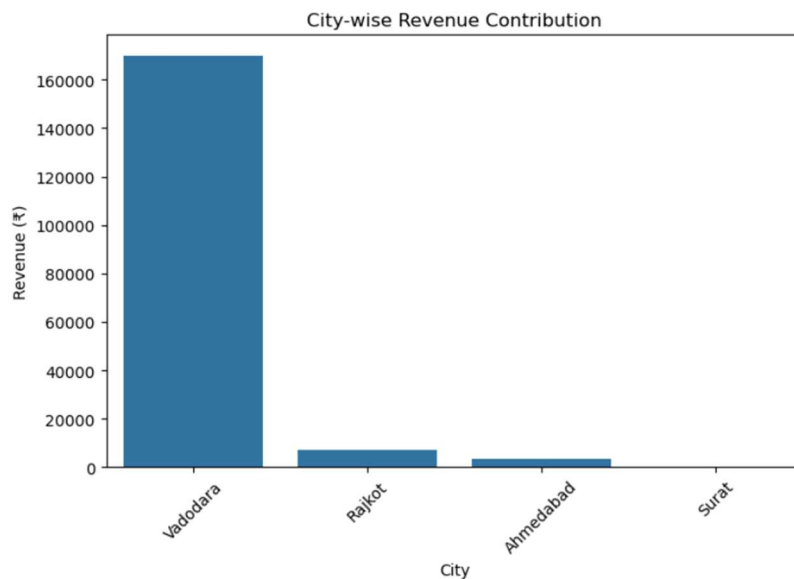
Output



Input Command

```
city_revenue = df.groupby(  
    'City'  
)['TotalAmount'].sum().sort_values(ascending=False)  
plt.figure(figsize=(8,5))  
sns.barplot(  
    x=city_revenue.index,  
    y=city_revenue.values  
)  
plt.title('City-wise Revenue Contribution')  
plt.xlabel('City')  
plt.ylabel('Revenue (₹)')  
plt.xticks(rotation=45)  
plt.show()
```

Output



Input Command

```
heatmap_data = pd.crosstab(  
    df['ProductCategory'],  
    df['PaymentMode']  
)  
plt.figure(figsize=(8,5))  
sns.heatmap(  
    heatmap_data,  
    annot=True,  
    fmt='d'  
)  
plt.title('Product Category vs Payment Mode')  
plt.xlabel('Payment Mode')  
plt.ylabel('Product Category')  
plt.show()
```

Output

